RELATÓRIO TÉCNICO

Aluno: Andrei Pereira
Disciplina: Sistemas Cognitivos com Large Language Models
Título do Projeto: Assistente de Suporte Técnico Baseado em RAG para Documentação Promob

1. Como reproduzir

1. Colab com GPU (T4 ou superior).
2. Colocar os arquivos .md na pasta /content/docs.
3. Executar a célula de instalação de dependências.
4. Ajustar parâmetros (modelo, chunk_size, top_k, estratégia de prompt), se necessário.
5. Executar main().
6. Ao trocar de configuração sem reiniciar o kernel, executar chromadb.api.client.SharedSystemClient.clear_system_cache() após apagar a pasta do vector store antigo, para evitar erro de banco somente-leitura.

2. Problema

A documentação de suporte técnico do software Promob contém linguagem técnica e muitas vezes não é bem interpretada pelos clientes. Pensando nisso desevolvi neste projeto um assistente de perguntas e respostas baseado na documentação de suporte.
A ideia inicial era um projeto que resumia as aulas do curso com base na transcrição, mas encontrei dificuldades usando recuperação por similaridade (top-k) o que não é adequado para um documento inteiro, e sim para trechos específicos. Por esta razão mudei o objetivo do projeto para um caso de uso mais compatível.

3. Documentos

Três artigos reais do suporte da empresa, convertidos para Markdown:

- Portal - Renovar Assinaturas
- Promob - Atualizar o Promob
- Promob - Requisitos do computador

São documentos curtos e de tópico bem definido, o que favorece a recuperação por similaridade.

4. Arquitetura

Documentos .md -> chunking -> embeddings -> ChromaDB -> retriever (top-k) -> prompt com contexto -> LLM local -> resposta

Stack: LangChain (LCEL), Hugging Face Transformers, ChromaDB. Execução local no Google Colab com GPU T4 (15GB de VRAM).

5. Modelos testados

Embeddings: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
LLM: Testado 3 opções:

- **mT5_multilingual_XLSum (580M):** descartado logo no início. É treinado exclusivamente para sumarização e não segue instruções: ao receber um prompt com instrução explícita, devolveu parte do próprio prompt como resposta.
- **LaMini-Flan-T5-248M (248M):** apresentou desempenho fraco em português. Nos testes, alternava para inglês no meio da resposta e, em algumas perguntas, recusou-se a responder citando uma
suposta "política de uso da OpenAI", que não tem relação nenhuma com este projeto.
- **mt0-xl (3.7B):** modelo escolhido para a versão final. Segue instruções de forma consistente e respondeu em português na maioria dos casos. Como contrapartida, exige bem mais VRAM (foi necessário carregá-lo em fp16 para evitar erro de memória)

6. Chunking e Retrieval

Os documentos são curtos, então um chunk desse tamanho deve ser suficiente:
RecursiveCharacterTextSplitter, chunk_size=500, overlap=50

Retriever: ChromaDB, busca por similaridade, top_k=3. Nas perguntas testadas, a resposta correta praticamente sempre estava concentrada em um único chunk, e o retriever conseguiu localizá-lo.

7. Execução

A execução foi local, sem uso de API paga. A vantagem neste contexto é o custo zero. Os documentos são públicos na página de suporte então privacidade não é uma vantagem neste caso.
A desvantagem é a latência. O modelo mt0-xl (3.7B) gerou um erro de memória (OutOfMemoryError) na primeira tentativa. A causa foi o modelo ter sido carregado em fp32 por padrão, consumindo cerca de 14.8GB. Alterando o carregamento para torch_dtype=torch.float16 o consumo de VRAM reduziu drasticamente.

8. Prompt Engineering

Todas com as mesmas 5 perguntas e o mesmo modelo (mt0-x1).

- **Zero-shot:** instrução direta, com regra explícita de recusa ("se não estiver no contexto, responda exatamente 'Não localizei essa informação no documento'").
- **Few-shot:** com de um exemplo de pergunta e resposta antes da pergunta real.
- **Structured output:** solicita a resposta em formato JSON: `{"resposta": "...", "encontrado_no_documento": true/false}`.

Perguntas utilizadas

Uma por documento, mais uma fora do escopo, para testar se o modelo alucina:

1. Como faço para renovar minha assinatura do Promob?
2. Quantos GB de RAM são recomendados para rodar o Promob?
3. É possível atualizar o Promob com o programa fechado?
4. O Promob funciona em máquina virtual?
5. Qual o preço da licença anual do Promob? (fora do escopo, não consta em nenhum documento)

9. Resultados

ZERO-SHOT (mt0-xl):

1. Correta: reproduziu os passos corretos do documento de renovação.
2. Incorreta: respondeu "4 GB - 8 GB", que corresponde ao espaço em disco, não à RAM recomendada (16GB). Houve confusão entre duas linhas da mesma tabela.
3. Correta no conteúdo ("Yes"), mas em inglês em vez de português.
4. Correta ("não").
5. Correta: recusou responder, sem inventar informação.

FEW-SHOT (mt0-xl):

1. Incorreta: respondeu com o procedimento de ATUALIZAÇÃO, não de renovação. O modelo parece ter reproduzido o conteúdo do exemplo usado no prompt (que também tratava de atualização) em vez de responder à pergunta real.
2. Mesma confusão entre RAM e espaço em disco.
3. Corretas no conteúdo, novamente em inglês.
4. Corretas no conteúdo, novamente em inglês.
5. Alucinou: respondeu "Gratuita", informação inexistente na base.

STRUCTURED OUTPUT (mt0-xl):
Nenhuma das 5 respostas veio em formato JSON válido; o modelo não seguiu o formato solicitado em nenhum caso. Na pergunta 5 (fora do escopo), alucinou o valor "$10,00".

LAMINI (248M), para efeito de comparação de modelo:
Testei zero-shot e few-shot também neste modelo. Em ambos os casos o resultado foi insatisfatório: alternância para inglês no meio das respostas, reprodução de trechos do próprio prompt como se fossem resposta, e citação de políticas que não existem neste projeto. A troca de técnica de prompt não alterou esse padrão, o que sugere que a limitação está no tamanho/capacidade do modelo, não na técnica utilizada.

10. Conclusões

- Apenas o zero-shot recusou corretamente a pergunta fora de escopo;
- O few-shot demonstrou que, se o exemplo usado no prompt for próximo de outras perguntas do domínio, o modelo pode reproduzir o conteúdo do exemplo em vez de aplicar o estilo na resposta;
- Nenhum dos modelos testados seguiu o formato JSON;
- O modelo maior (mt0-xl, 3.7B) superou o menor (LaMini, 248M) em qualidade e consistência no idioma;
- Uma falha constante: a pergunta sobre RAM sempre retornava o valor errado (espaço em disco)
- Chunking genérico não lida bem com tabelas Markdown (causou o erro de RAM x espaço em disco)